

AUVRA Architecture and Operations Handbook

Initial v2 rebuild handover package | generated evidence snapshot | 2026-08-08

Important: AS-IS is frozen evidence. TARGET PLANNED/PARTIAL items are designs or incomplete work, not production claims. Legacy restore is blocked pending an isolated PostgreSQL 17 rehearsal; the supplied 22-byte storage ZIP is empty.

Contents

1. 00 Founder Summary
2. 01 As Is And Legacy
3. 02 Target Architecture
4. 03 Data And Metrics
5. 04 Api And Mobile
6. 05 Security Privacy
7. 06 Delivery And Acceptance
8. 07 Traceability
9. 08 Handover And Change Control
10. Adr 001 Modular Monolith
11. Adr 002 Identity
12. Adr 003 Onboarding Consent
13. Adr 004 Plans Actions Reviews
14. Adr 005 Time Streak Rewards
15. Adr 006 Conversations Checkins
16. Adr 007 Durable Jobs Outbox Idempotency
17. Adr 008 Ai Evidence Safety
18. Adr 009 Media
19. Adr 010 Metrics
20. Adr 011 Api Contract
21. Adr 012 Migrations Environments
22. Adr 013 Health Data Lifecycle
23. Adr 014 Observability Operations
24. Adr 015 Mobile Architecture
25. Api_And_Database
26. Jobs_And_Providers
27. Restore_And_Migration
28. Security_Privacy
29. Under_Pressure
30. Migration Evidence

AUVRA v2 in fifteen minutes

The outcome

AUVRA v2 is a controlled rebuild around one source of truth per business concept. The target is a FastAPI modular monolith plus one separately deployed durable worker, PostgreSQL 17 as the business authority, Redis only as queue/cache/lock infrastructure, and an iOS/Android React Native client using a single generated OpenAPI client.

This package is an initial design and handover baseline, not evidence that v2 is complete. Written owner approval is pending. Production release is blocked until architecture, legal, clinical, privacy, migration and readiness gates are recorded as passed.

Why the rebuild is controlled

The frozen backend commit contains 69,102 application Python lines, 35 ORM tables in `app/core/database.py`, 132 detected HTTP operations, four `asyncio.create_task` call sites and 146 direct `.commit()` call sites. The largest generator is 9,529 lines. Static route evidence identifies 52 operations without a recognized in-function authentication guard, including 19 mutations; this is a conservative lexical signal requiring manual review, not a complete authorization proof.

The frozen mobile commit contains 56,708 tracked TypeScript/JavaScript lines, a 3,599-line chat screen, a 3,103-line home screen, 61 raw fetch calls across 14 files, 12 files implementing API URL logic, no detected test files, seven source references to password persistence through `AsyncStorage`, and temporary/old source copies. These measurements are reproducible in the evidence snapshot.

The existing deployment and schemas are therefore restricted reference evidence, not an approved rollback destination once v2 writes begin.

The target shape

```
iOS / Android
|
| OpenAPI v2 + verified Firebase token
v
FastAPI API process -----> PostgreSQL 17 (app + ops + runtime)
| ^
| enqueue identifier | durable job state / atomic publication
v |
Redis queue -----> Durable worker -----> AI/evidence/image adapters
|
+-----> permanent object storage/CDN
```

The API and worker use one immutable image. The API owns synchronous adaptation; the worker owns durable provider orchestration. Only PostgreSQL owns business state. Provider calls never occur inside an open database transaction.

Eight modules, one owner each

1. Identity and access. 2. Onboarding, consent and profiles. 3. Plans, actions and Daily Review. 4. Progress, streaks, rewards and refreshes. 5. Conversations and check-ins. 6. Symptom observations. 7. Evidence, AI safety and evaluation. 8. Media, jobs and operational infrastructure.

Each module follows API adapter -> application command/query -> domain policy -> repository/provider port. Cross-module callers use application interfaces, not another module's database implementation.

Non-negotiable data rules

- Private identity comes only from a verified Firebase token mapped to internal `app.users.id`; a request-body `user_id` never authorizes access.
- Guest onboarding uses a hashed proof credential distinct from its session ID.
- All timestamps are UTC `TIMESTAMPZ`; daily decisions also store the immutable IANA timezone and local business date used.
- A plan revision is `READY` only when one transaction publishes four valid actions and sixteen reachable permanent images: hero plus three variants for each action.
- Completion, skip and feedback facts are immutable and replay safe.
- Streak days and reward movements are ledgers; current values are calculated.
- LangGraph checkpoints are expiring runtime state, never conversation or health-data truth.
- Deletion covers Firebase, PostgreSQL, storage, conversations, checkpoints, caches and derived summaries.

The critical journey

validated assessment/profile -> context snapshot -> evidence retrieval -> structured model output -> citation and safety validation -> image generation -> permanent storage -> atomic plan publication -> asynchronous evaluation

PostgreSQL records every durable job state. Redis only transports a job ID. A worker claims a lease, heartbeats, retries within an attempt limit and dead letters exhausted work. Blank/temporary images or unsafe/unverifiable content cannot be published.

Safe mobile mental model

Firebase owns login state. The generated OpenAPI client is the only place that may resolve an API URL, retrieve a token or call fetch. TanStack Query owns server state; small feature reducers/state machines own ephemeral workflow state. SecureStore contains only necessary credentials, never passwords or durable health drafts. Logout/account switching clears query state and all UID-scoped local data. A plan remains hidden until all hero images are prefetched through one PlanImage component built on expo-image.

What is not decided or verified

- Owner approval of the retained-feature matrix, table mapping, metrics, API and ADRs is pending. - Jurisdiction, consent wording, retention duration, health-product classification and clinical escalation text are pending legal/clinical owner decisions. - No provider/model/image route is production-approved by this document; selection requires rotated credentials and staged safety/quality/cost/latency benchmarks. - No production SLO, backup objective, security posture or performance target is claimed met. Listed values are initial acceptance targets. - The backup contains exactly 1,022 application COPY rows plus one alembic_version row, but an isolated PostgreSQL 17 restore has not been rehearsed: local client tools are PostgreSQL 14 and Docker is unavailable. This is a release blocker. The supplied storage ZIP is a valid empty 22-byte archive, so no legacy media can be restored; retained images must be regenerated and permanently persisted.

How to make a change without creating another source of truth

Start at the feature record. Identify its owning module, API operation, canonical table and governed metric. Amend the relevant ADR when a decision changes. Update OpenAPI, ORM, Alembic, catalogs, ERD and tests in the same change. Validate blank bootstrap and rollback. Add telemetry and a runbook before adding an alert. A second active table, calculation, API DTO or cache claiming the same concept is a release-blocking design defect.

When the system is under pressure, start with runbooks/UNDER_PRESSURE.md and make read-only checks before intervention.

AS-IS evidence and legacy disposition

Evidence boundary

AS-IS statements in this package are tied to committed objects, not a mutable working tree:

Repository	Frozen commit	Measured scope
Backend	c02892a8e1d5ec6bc76040e25df4e93d8fb60cfc	Tracked source at commit, primarily <code>app/*.py</code>
Mobile	923e09c234c7b105d9f9ec15347ea55fa45a104	Tracked TS/TSX/JS/JSX excluding native build trees and dependencies

The exact counts, largest-file list, table list, route scan methodology and selected source hashes are in `evidence/repository_snapshot.json`. The route authentication count is a static signal only: it detects recognized guards inside operation functions and cannot prove all dependency injection or object authorization behavior. Every private object still needs an explicit cross-user test.

Observed architecture

The backend is one FastAPI process exposing `/api/v1`, with endpoint modules that call a large set of services and SQLAlchemy models. All 35 application ORM tables are declared in one 1,614-line database file. Direct transaction calls are distributed throughout application code. Firebase initialization can continue after failures, production host defaults allow *, CORS allows all origins, and some background generation uses `in-process asyncio.create_task`. Redis appears for rate limiting/cache behavior, while LangGraph checkpoint configuration is mixed into the same runtime.

The mobile application starts from `App.tsx` with React Navigation, but still declares Expo Router and web dependencies/configuration. API calls, token logic and URL selection are duplicated across services and screens. `AsyncStorage` holds workflow state, assessment/review drafts and, in the legacy auth service, password-related values. Large screens and temporary copies compete as implementation references.

Systemic risks evidenced by source

Risk	Evidence	v2 control
Lost asynchronous work on process restart	Four committed <code>asyncio.create_task</code> call sites	Durable PostgreSQL job record, Redis transport, worker lease/heartbeat/dead letter
Ambiguous identity/authorization	Static scan finds 52 operations without a recognized local guard	Token-derived internal user plus repository owner predicates and cross-user tests
Partial or broken plans	Generation and image work span large services and mutable status	Atomic READY publication after four-action/sixteen-image validation
Competing business truth	35-table model contains recommendation, assignment and plan families plus counters/arrays	Canonical v2 ledgers and plan family; legacy archive is non-serving
Transaction drift	146 direct <code>.commit()</code> calls found in tracked Python	One SQLAlchemy 2.x Unit of Work owns <code>begin/commit/rollback</code>
Sensitive device storage	Password, assessment and Daily Review references in <code>AsyncStorage</code>	Firebase session truth, <code>SecureStore</code> only for necessary credentials, server-side/time-limited drafts
Mobile contract drift	61 raw fetch calls and 12 API URL implementations	Generated OpenAPI client is the sole network/token/URL layer
Unverified behavior	No detected mobile test file	Component and Maestro device tests on both platforms

Legacy database boundary

`docs/DATABASE_RECOVERY.md` remains the authoritative historical recovery procedure for the 35-table public-schema baseline. The supplied backup's COPY sections contain the following exact application row counts:

Table	Rows	Table	Rows
daily_assignments	145	question_sessions	4
recommendation_advices	498	recommendation_completions	1
recommendation_records	169	recommendation_redistributions	0
recommendation_schedules	141	schedule_redistributions	5
session_processing_status	8	user_profiles	25
user_responses	25	user_schedules	1
Application total	1,022	alembic_version (separate)	1

Its safe principle is retained: restore the cluster dump only into an isolated PostgreSQL 17 source, bootstrap a blank target through Alembic, and copy only explicitly reviewed application columns.

That runbook bootstraps the legacy 35-table schema; it is not the v2 serving schema and its existing copier must not be presented as a completed v2 migration. The v2 migration requires UUID mapping, row-level disposition, quarantine reports and object hashing against the new app/ops schemas. Restore rehearsal is not verified: local pg_restore/psql are PostgreSQL 14 and the Docker daemon is unavailable, so the required isolated PostgreSQL 17 restore and two target rehearsals remain release blockers.

The supplied storage ZIP is a valid but empty 22-byte archive (unzip -t reports an empty archive). No legacy media can be restored from it. Required plan media must be regenerated through approved providers/fallbacks, permanently persisted, hashed and reconciled before any plan is published READY.

Disposition summary

- question_sessions and user_responses become one guest workflow plus one canonical validated assessment. - Legacy recommendation/scheduling/assignment tables are archived. Only completion/skip history with deterministic user, item and time mapping may become action events.
- user_profiles splits identity from mutable profile; chatbot memory, counters and scheduling fields lose source-of-truth status.
- Chat, care-plan and symptom threads consolidate into typed conversations, normalized messages and versioned summaries. raw_messages arrays disappear.
- The action-plan family is rebuilt with UUIDs, revisions, constraints and permanent media foreign keys. Feedback and reviews become immutable events and normalized review rows.
- Mutable streak/reward/refresh counters and date arrays become ledgers.
- Mood tracking is archived by default because it is inactive; mock, test, backup and unreachable features are removed unless the owner explicitly retains them.

Every individual legacy table, target table, row grain and mapping is recorded in catalogs/tables.json. Ambiguous data is archived or quarantined; it is never guessed into a canonical v2 row.

Legacy rollback statement

Before v2 receives writes, a cutover can be abandoned while the legacy system remains restricted. After meaningful v2 writes, rollback is only to a compatible v2 application/schema or a tested v2 restore. The insecure legacy deployment is not an approved production rollback target.

TARGET architecture

Governing pattern

AUVRA uses a modular monolith because the domain boundaries need to stabilize before operationally independent services would be justified. One application image produces two deployables: the synchronous FastAPI API and a durable worker. This follows the readiness tradeoff described in Microsoft's microservices assessment guidance.

The system context, container, backend component, mobile component and deployment views live under diagrams/c4/. Mermaid is the editable diagram source; the handbook builder embeds vector renderings or a vector fallback and records the renderer in the manifest.

Implementation boundary observed on 2026-08-08

The current workspace contains 20260801_0002_auvra_v2_foundation.py, a partial target-schema migration with twelve catalogued app/ops tables (identity, onboarding, jobs, outbox, idempotency, media and plan foundation). It does not yet create the remaining eighteen catalogued app/ops target tables, and no isolated PostgreSQL 17 upgrade/rollback evidence is recorded. This is TARGET PARTIAL, not a deployed schema or a verified foundation. The catalog continues to use TARGET PLANNED for the intended complete table contract; the release validator reports the implemented subset separately.

Backend dependency rule

Every module follows:

API adapter -> application command/query -> domain policy -> repository/provider port

Only infrastructure adapters implement SQLAlchemy repositories, provider SDKs, Redis transport, storage or telemetry. The same SQLAlchemy 2.x session factory and Unit of Work are used across modules. Only the Unit of Work opens, commits or rolls back a transaction. A provider call occurs outside database transactions; state is first committed as a durable job/outbox fact and completed through a later short publication transaction.

Module ownership

Module	Owns	May consume through
Identity and access	Internal user mapping, authentication policy	Firebase identity port
Onboarding, consent and profiles	Guest workflow, assessment, consent, profile	Identity application query
Plans, actions and Daily Review	Plans, items, variants, events, reviews, refresh operations	Profile snapshot, evidence and media ports
Progress, streaks, rewards and refreshes	Streak and reward ledgers, progress queries	Immutable plan/action/review facts
Conversations and check-ins	Typed threads, messages, summaries, weekly check-ins	AI gateway, profile/plan read models
Symptom observations	Structured symptom facts	Conversation commands and profile identity
Evidence, AI safety and evaluation	Sources, citations, invocation metadata, evaluation policy	Provider gateway ports
Media, jobs and operational infrastructure	Media, durable jobs, outbox, idempotency and audits	Provider/storage/queue adapters

No module imports another module's SQLAlchemy models/repositories. A cross-domain change uses an application interface and an outbox event when temporal decoupling is useful.

Runtime responsibilities

PostgreSQL is authoritative for application and operations data. Redis transports job identifiers, caches recomputable results and supports locks; eviction cannot erase business truth. LangGraph's four checkpoint tables live in runtime, are vendor-owned, have explicit retention and never substitute for messages, summaries or observations.

The worker claims jobs with FOR UPDATE SKIP LOCKED or an equivalently tested lease algorithm, increments attempt state, heartbeats, releases to retry_wait with bounded backoff, and ends exhausted work in dead_letter. Redis delivery is at least once; handlers are idempotent.

Configuration and promotion

- Local/CI, staging and production use separate Supabase, Firebase, Redis, storage and provider credentials. - Production typed configuration is fail closed. Placeholder/missing identity or database credentials, wildcard origins/hosts or development authentication prevent startup. - Database migrations run once before deploy, never on every API/worker startup. - The identical immutable image is promoted from staging to production. - Supabase Data API exposure excludes private application schemas; mobile reads health data only through the backend. - Company infrastructure and signing accounts have company ownership and at least two administrators.

Architecture fitness rules

CI rejects provider SDK imports outside adapters, transaction calls outside the Unit of Work, request-handler fire-and-forget work, module-to-module database imports, raw mobile fetch, direct AsyncStorage health data, unauthenticated private routes and catalog/ORM/Alembic/OpenAPI drift.

Handwritten files above 800 lines, functions above 100 lines or cyclomatic complexity above 15 fail unless an ADR documents a narrowly scoped exception. Generated source and vendor-owned migrations are classified separately.

Status

This is a TARGET PLANNED architecture. Presence in this handbook does not prove the rules are implemented. Verification requires the phase gates in 06-delivery-and-acceptance.md and dated evidence in the build manifest/release record.

Canonical data and metric governance

Schema ownership

The v2 serving database has three private schemas:

- app contains application truth owned by the eight domain modules. - ops contains durable jobs, outbox, idempotency and redacted audit events. - runtime contains the four vendor-owned LangGraph checkpoint tables with retention rules.

Legacy rows and objects stay in an encrypted, access-restricted archive or isolated migration source. No legacy table remains in the serving schema. The logical ERDs are in diagrams/data/; a physical appendix must be regenerated from the final Alembic/ORM state before release.

Canonical row families

Family	Canonical grain
Identity/onboarding	Internal user; mutable profile; immutable consent decision; guest session; validated assessment
Plans	Plan revision per user/local date; item per slot; typed variant per item
History/review	Immutable action event; Daily Review per plan; answer per reviewed item; accepted refresh operation
Engagement	Adjudicated streak day; immutable reward-ledger movement
Conversation/check-in	Typed conversation; ordered message; versioned summary; weekly check-in/question/response
Symptoms	Timestamped structured user-owned observation
Media/evidence/quality	Immutable media version; research source; claim-source link; provider invocation metadata; plan evaluation
Operations/runtime	Durable job; outbox event; idempotency result; redacted audit event; vendor checkpoint row

The table catalog gives all 34 target/runtime tables and maps every one of the 35 legacy ORM tables.

Enforced invariants

1. Every user-owned row has an internal user foreign key with explicit deletion behavior. 2. UTC TIMESTAMPTZ records instants; daily policy also stores immutable IANA timezone and local date. 3. Assessment ownership is exactly one guest session or one user. 4. Plan (user_id, local_date, revision) and one-current-plan constraints are explicit; revisions are preserved. 5. Item slot is unique per plan and variant type is unique per item. 6. READY publication atomically proves four actions and sixteen reachable permanent images. 7. One Daily Review exists per plan and one answer per plan item. 8. Client operation/message identifiers and idempotency keys make retries safe. 9. Refresh count, reward balance and streak values are queries over immutable facts, never second counters. 10. No health-data row is ownerless or lacks an approved archival class.

Some invariants require deferred database constraints or publication-domain checks because they span multiple rows and external object reachability. They must still be transactionally fail closed and covered by reconciliation tests.

Metric ownership

Metrics are definitions over canonical fields, not columns copied into unrelated tables. catalogs/metrics.json records formula, grain, timezone, exclusions, source fields, owner, freshness and consumers for:

- plan-generation success; - ready-plan completeness; - action completion and daily adherence; - current streak and refreshes used; - weekly check-in completion; - AI cost per ready plan; - core API availability; and - Daily Review persistence.

A projection/cache is allowed only with a named owner, documented lineage, refresh policy and a reconciliation test against canonical facts. The current day may show a clearly labeled provisional streak/adherence value but cannot be finalized before the user-local day closes.

Health-data lifecycle

Collection must be purpose-limited and consent-versioned. Encryption, access control, production telemetry redaction, export and complete erasure apply to answers, messages, summaries, observations and derived results. Exact retention durations and any legally required consent-record retention are unresolved owner/ legal decisions; the application must fail release, not invent a duration.

Migration proof

For each of the known 1,022 legacy application rows (plus the separate migration version row), the v2 migrator produces exactly one of:

- a target table and stable legacy-ID-to-UUID mapping; - an approved archive class; or - a quarantine record with reason and no serving import.

Object migration records content SHA-256 and classifies referenced, duplicate, orphaned and missing objects. Two clean target rehearsals must produce identical counts/hashes, zero orphan foreign keys, zero duplicate business keys and zero broken retained assets.

Current evidence does not satisfy that gate. The supplied storage ZIP is a valid empty 22-byte archive, so there are no recoverable legacy media objects; images must be regenerated and stored permanently. PostgreSQL 17 restore rehearsal is also blocked locally because only PostgreSQL 14 client tools are installed and the Docker daemon is unavailable.

Public interface and mobile integration

Contract

The target contract is checked-in OpenAPI 3.1.1 with stable operation IDs and RFC 9457 problem details. The API catalog records 26 proposed operations. It is the design baseline; implementation and generated-client drift tests must still prove the checked-in OpenAPI matches runtime behavior.

Private operations derive identity exclusively from the verified Firebase token. Object access applies both the internal user predicate and resource relationship in the repository query. Request-body/path user_id values never authorize. Session-specific onboarding operations require a separate guest proof credential.

Every mutation requires Idempotency-Key; retries with the same key and request hash replay the original result, while a different hash returns a stable conflict problem. Plan mutations also require If-Match revision validation. Errors return application/problem+json, stable code, safe detail and correlation ID; they never return stack traces, provider exceptions, tokens or prompt contents.

Native plan representation

The ActionPlan DTO contains plan ID, revision, local date, timezone, cycle snapshot, four canonical items, completion summary and typed image assets. The client never reconstructs a plan from assignments or merges competing legacy DTOs. ETags represent resource revisions.

Generation behavior

POST /api/v2/plan-generations commits a durable job and returns its ID. The client observes GET /api/v2/jobs/{job_id} states:

queued -> running -> retry_wait -> running -> ready

or a terminal failed, cancelled or dead_letter. A ready response links to the published plan. Timers and AsyncStorage flags never decide job truth.

Mobile layering

```
feature screen/container
-> feature hook + reducer/state machine
-> generated OpenAPI client + TanStack Query
-> auth/token + URL + transport core
```

Raw fetch, Firebase token lookup and base URL resolution exist only in the client transport. Server DTOs come from generation, not copied interfaces. React Navigation remains; Expo Router and web-only configuration are removed. The supported release platforms are iOS and Android.

Firebase listener state is the only login-session truth. SecureStore keeps only necessary credentials; passwords are never stored. Health drafts are server-side or encrypted with a documented TTL. Logout/account switch cancels requests, clears TanStack Query, removes secure credentials and deletes all UID-scoped local/cache data before the next account renders.

One PlanImage component uses expo-image. All four hero images are prefetched and verified before the plan is revealed. Permanent approved fallbacks are category-specific; blank or temporary URLs never render a READY plan.

Migration order

1. Generated client, auth/bootstrap, typed navigation and storage/query core. 2. Onboarding and claim. 3. Plan, action event, review and progress. 4. General conversation and typed check-ins. 5. Profile/export/deletion and any owner-approved remaining feature. 6. Remove mock screens, duplicate DTOs, fake plans, dead services, temporary copies, debug signing assets and unused web/router dependencies.

Each journey passes component tests and Maestro E2E on iOS and Android, including token expiry, duplicate taps, process death, offline recovery, account switch, timezone/DST boundaries, provider/image failure and accessibility.

Security, privacy and trust boundaries

Boundary model

The mobile device, public internet, API/worker runtime, managed data stores and external providers are separate trust zones. The trust-boundary diagram is `diagrams/c4/trust-boundary.mmd`. The API validates Firebase tokens and maps the subject to one internal user before any private query. Workers accept only server-issued job IDs and re-read authoritative state from PostgreSQL.

Data classes

Class	Examples	Production handling
Credentials/secrets	Firebase tokens, provider keys, DB URLs, signing keys	Secret manager/environment only; never logs, Git or catalogs; rotate on exposure
Direct identifiers	Email, provider subject, internal user ID	Least privilege; internal ID in operational joins; redact external identifiers from telemetry
Health-related content	Assessment answers, symptoms, messages, summaries, plan context	Purpose/consent limited, encrypted, owner-bound, omitted from telemetry, export/erasure covered
Derived health content	Plans, citations, evaluations, adherence/streak facts	Same lifecycle protection as source health data unless approved otherwise
Operational metadata	Job state, latency, tokens/cost, error code, correlation ID	Redacted; no prompt, email, answer, token or conversation content
Public evidence/media	Verified research metadata, approved category fallback	Integrity/version checks; still no user identifiers in provider prompts

Top threat paths and controls

Threat	Prevent/detect control	Release evidence
Cross-user object access	Token-derived internal user plus repository owner predicate	Authorization matrix for every private object and hostile ID substitution
Guest-session takeover	High-entropy proof stored hashed, short TTL, rate limit, single claim	Guess/replay/expired/concurrent claim tests
Mutation replay/race	Request hash + Idempotency-Key + unique client operation + revision ETag	Duplicate tap, timeout retry and conflicting payload tests
Worker loss/duplication	PostgreSQL job authority, leases, heartbeats, bounded retry, idempotent publication	Kill/restart, lease expiry and dead-letter tests
Unsafe/unverifiable plan	Structured output, authoritative citations, deterministic and model safety gates	Clinical red-team fixtures and fail-closed publication tests
Sensitive logs/traces	Allowlisted telemetry attributes and logger redaction	Canary-secret/health-string scan across logs/traces/errors
Mobile account bleed	UID-scoped cache plus complete logout/account-switch purge	Two-account device E2E and storage inspection
Incomplete erasure	Deletion step ledger across identity, DB, objects, checkpoint/cache/summary	Post-deletion enumerator proves zero user-scoped remnants
Supply-chain compromise	Locked dependencies, SBOM, SAST/dependency/container/mobile scans	Zero open Critical/High findings and signed immutable image evidence

Account erasure

DELETE /api/v2/me uses recent authentication and returns a durable deletion job. The worker marks the account deletion-pending, revokes/blocks new access, removes objects and runtime/checkpoint/cache state, deletes or legally isolates database rows, deletes the Firebase identity, and writes a content-free completion receipt. Each step is idempotent and restartable. Partial failure remains visible and alerted; the API never claims completion until the verification enumerator finds no disallowed remnants.

The exact retention exception policy is unresolved and requires legal/owner approval. A retained legal record must be minimal, access-restricted and detached from serving use; this handbook does not declare that any exception applies.

Release security gate

Use NIST SSDF, OWASP API Security Top 10 and OWASP MASVS as control frameworks, plus a repository threat-model review. Production requires zero open Critical or High security findings, no unresolved P0/P1 defect, rotated credentials and a dated account-export/erasure test. These are gates, not current claims.

Delivery sequence and acceptance evidence

Current program state

This initial package completes part of the Phase 1 alignment material. It does not assert that Phase 0 evidence preservation, credential rotation, infrastructure restriction, target implementation, migration or staging gates have passed. Owner approval remains pending.

Phase gates

Phase	Exit evidence required	Status at this release
0 Preserve/contain	Tags, checksummed DB/storage/config/flow evidence, verified restore, rotated credentials, restricted legacy	Blocked: PG17 restore not verified; supplied storage ZIP is empty
1 Align	Approved feature/table/metric catalogs, ERDs, API and ADRs; no P0/P1 decision open	Package produced; approval pending
2 Foundation	Blank DB bootstrap/rollback, UoW, auth, idempotency, worker restart and transaction tests	Not evidenced complete
3 Backend slices	Contract, ownership, migration, failure, telemetry and domain tests per slice	Not evidenced complete
4 Mobile integration	Retained iOS/Android journeys, generated client, no raw fetch/plaintext password/cross-account cache	Not evidenced complete
5 Migration	Two identical clean rehearsals, 1,022-row disposition, regenerated asset inventory, zero FK/key/asset defects	Blocked: no local PG17/Docker rehearsal and no restorable legacy media
6 Staging readiness	Exact artifacts, full tests, 72-hour soak, independent runbook exercise, zero Critical/High or P0/P1	Not evidenced complete
7 Cutover/hypercare	Final reconciliation, smoke tests, two-hour watch, seven days without P0/P1, restore evidence	Not evidenced complete

Automated evidence suites

- Domain property/state tests for plan invariants, idempotency, timezone/DST, streak/freeze, review and metric formulas. - PostgreSQL/Redis integration; blank/upgrade/rollback; schema drift. - OpenAPI/runtime/generated-client drift. - Authentication and cross-user authorization for every private object. - Worker kill/restart, lease expiry, timeout, retry and dead-letter behavior. - React Native components and Maestro E2E on physical/simulator iOS and Android. - Secret, dependency, SAST, container, API and mobile-storage scans. - Export/deletion verification across Firebase, DB, objects, summaries, checkpoints and cache.

Initial targets - not current claims

SLI	Initial acceptance target
Core synchronous API availability	At least 99.9%
Non-AI latency	p95 under 750 ms; p99 under 2 s
Current-plan retrieval	p95 under 1 s
Valid plan generation	At least 98%; p95 under 120 s
READY plan completeness	100% four actions and sixteen reachable permanent images

SLI	Initial acceptance target
Daily Review persistence	At least 99.9%; zero duplicate keys
Authorization/data integrity	Zero cross-user access and zero orphan records
Capacity floor	25 interactive users plus five generation jobs for 30 minutes within SLOs
Recovery	Tested RPO <= 1 hour; RTO <= 2 hours
Coverage	Critical domain/auth/deletion >= 90% branch; backend >= 80%; mobile >= 75%

OpenTelemetry provides vendor-neutral traces, metrics and logs. User-relevant SLI/ SLO design follows Google SRE principles. Health answers, prompts, emails, tokens and conversation contents are forbidden from telemetry. An SLO without instrumentation, an alert without a runbook or a production-ready claim without dated evidence fails release review.

Cutover and rollback

Freeze legacy writes, take final backups, migrate/reconcile, deploy the exact staging-proven API/worker artifact, smoke-test authenticated flows and switch the stable company hostname. Watch continuously for two hours, then through seven days of hypercare. After v2 writes, rollback only to compatible v2 code/schema or a verified v2 restore; never direct traffic back to the insecure legacy service.

Feature-to-screen-to-API-to-table-to-metric traceability

This is the human-readable alignment matrix. The complete machine form is catalogs/features.json; approval remains pending.

Feature	Disposition	Mobile surface	v2 operation(s)	Canonical truth	Governed metric
Identity/session	REBUILD	Splash, Login, SignupLoading	profile, delete-me	users, profiles, audit events	API availability
Guest onboarding/claim	REBUILD	Onboarding, Question, Result, Researching	onboarding session/assessment/claim	onboarding sessions/assessments, consent	plan-generation success (downstream)
Daily plan	REBUILD	Home, ActionDetail, Researching	plan generation, job, today/get plan	plans/items/variants, generation jobs	generation success, completeness, AI cost
Action completion/feedback	REBUILD	Home, ActionDetail, ActionCompleted	item event, feedback	immutable action events	action completion, daily adherence
Daily Review	REBUILD	Home, DailyReview modal	put Daily Review	reviews/review items, idempotency	adherence, review persistence
Replacement/streak/reward	REBUILD	Home, Personalize, Progress	replacement, progress summary	refreshes, streak days, reward ledger	refreshes used, current streak
General conversation	REBUILD	Chatbot, ChatHistory	conversations/messages	conversations/messages/summaries	API availability
Weekly check-in	REBUILD	Chatbot, Home	due/create/respond	check-ins/questions/responses + conversation	weekly completion
Care-plan check-in	REBUILD	Chatbot	typed conversation /messages	conversations/messages/summaries	API availability
Symptom check-in	REBUILD	Chatbot, Home	symptom observation + typed conversation	symptom observations + conversations	API availability
Profile/export/erase	REBUILD	Profile	get/patch profile, export, delete	users/profiles, durable jobs/audits	API availability
Insights	REBUILD only after approval	Insights	No approved v2 endpoint yet	symptoms/action events	No approved metric yet
Mood tracking	ARCHIVE	Inactive component	None	None in serving schema	None
Paywall/community/test screens	REMOVE by default	Mock/test/unapproved surfaces	None	None	None
Web client	REMOVE	Web-only branches/config	None	None	None

Trace rule

A retained screen must point to exactly one generated client operation. That operation points to one owning application command/query. Writes end in one canonical table family plus idempotency/outbox infrastructure. Displayed product numbers point to one metric record. If two active paths claim the same concept, the change is blocked until one becomes the canonical owner and the other is removed or documented as a reconciled projection.

Handover and safe change control

First-day exercise

A new engineer should be able to explain the system context, locate an owning module from a feature, trace its screen to operation/table/metric, start API and worker, bootstrap a blank database, run upgrade and rollback, diagnose an injected provider/job failure and outline a schema/API/metric change without creating competing truth. Until the implementation and runbooks make that exercise pass independently, handover is incomplete.

Change recipe

1. Find or add the feature record and confirm its owner/disposition. 2. Identify the owning module; do not import another module's repositories. 3. Amend an ADR when the governing decision changes. 4. Update OpenAPI operation/schema, generated client and contract tests together. 5. Update ORM, a single Alembic descendant, table catalog and ERD together. 6. Update metric formula/source/consumer and reconciliation tests when a display or SLO changes. 7. Add domain/property, authorization, idempotency, failure and telemetry tests. 8. Add/update the alert and linked runbook together. 9. Prove blank bootstrap, upgrade and rollback in PostgreSQL 17. 10. Record dated evidence; never label a target verified from code presence alone.

Review questions

- Which single component owns this concept and its lifecycle? - Can a retry, duplicate delivery, worker death or account switch change the outcome incorrectly? - Is identity derived from the verified token and constrained in the repository query? - Does any provider call occur with a transaction open? - Could telemetry, a provider request or device storage expose health content? - Is local-date behavior deterministic across timezone changes, midnight and DST? - Is deletion/export coverage updated? - Does migration classify every source row/object without guessing? - Does the alert point to an independently exercised runbook?

Decision authority

Product scope, jurisdiction, consent wording, retention and health-product/ clinical classification require recorded owner/legal/clinical authority. Architecture owners may propose but not invent those decisions. Provider/model selection requires a staged benchmark and privacy/safety review. Schema and contract decisions require the affected domain owner plus architecture review.

Documentation release

The PDF builder validates JSON catalogs, records source hashes, generates the HTML companion and PDF, checks metadata/bookmarks/links and mandatory content, runs pdfinfo, renders every page with Poppler and produces contact sheets for visual review. The manifest and SHA-256 file bind the released artifact to its sources. A visually clean PDF does not convert planned controls into verified ones; status labels remain evidence-driven.

ADR-001: Modular monolith with one durable worker

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: Architecture owner; Platform engineering

Context

The legacy backend has intertwined plan, scheduling, conversation, AI and engagement implementations. Splitting unstable boundaries into microservices would add deployment, data-consistency and observability costs before ownership is clear.

Decision

Build one FastAPI modular monolith with eight enforced modules and one separately deployed durable worker from the same immutable image. PostgreSQL is the business authority. Modules expose application interfaces and cannot import another module's database implementation. The API performs synchronous adaptation; the worker performs durable asynchronous orchestration.

Consequences and verification

Deployment remains simple while module boundaries become testable. A future service extraction requires its own ADR and evidence that independent scaling or ownership outweighs distributed-system cost. CI checks module imports, provider SDK boundaries, transaction ownership and file/complexity limits. Staging proves the same image can run API and worker and survives worker restart.

ADR-002: Internal identity mapped from verified Firebase subjects

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: Identity and access

Context

Legacy routes and tables use Firebase UIDs directly and some routes accept user identifiers in paths/bodies. This weakens ownership boundaries and complicates provider change or account lifecycle.

Decision

Firebase remains authentication-session truth. Every private request verifies the token and maps (provider, subject) to one internal UUID app.users.id. Request data never authorizes access. Repositories include the internal user predicate in object queries. Disabled/revoked/deleted accounts fail closed. Mobile never stores passwords.

Consequences and verification

Application foreign keys become provider-independent and cross-user tests become uniform. Claim, export and deletion require recent-auth policies. Verification includes expired/revoked tokens, subject collision, hostile object IDs, account switch and Firebase deletion behavior.

ADR-003: Proof-bound guest onboarding and versioned consent

- Status: Proposed - written owner/legal approval pending - Date: 2026-08-01 - Owners: Onboarding, consent and profiles

Context

Guest sessions contain health-related answers before authentication. Session or device identifiers alone are not authorization, and copying answers during claim creates duplication.

Decision

Create an expiring guest session with a high-entropy proof returned once and stored only as a hash. One validated/versioned assessment is owned by exactly one guest session or user. Claim atomically transfers ownership after Firebase and proof verification. Consent decisions are immutable and reference document version, jurisdiction/purpose metadata and time.

Consequences and verification

Expired, guessed, replayed and concurrent claims fail safely. Assessment answers are not copied. Exact consent wording, jurisdiction and retention remain release- blocking owner/legal decisions. Tests cover proof rotation/expiry, simultaneous claim, orphan prevention and deletion/export.

ADR-004: Revisioned plans and immutable action history

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: Plans, actions and Daily Review

Context

Legacy recommendation, assignment and plan systems compete, while feedback and reviews mix mutable flags and JSON arrays.

Decision

One plan revision exists per user/local date/revision, with one current revision. Each plan has unique slots, typed variants and replacement lineage. Completion, skip and feedback are immutable events with client operation IDs. One Daily Review header per plan has one normalized row per item. A short publication transaction marks READY only after four actions and sixteen permanent reachable images are validated.

Consequences and verification

History remains explainable and replay-safe; current state is derived. The READY invariant spans rows and object reachability and therefore needs domain validation plus reconciliation. Property tests cover revision races, duplicate events, replacement lineage, partial image failure and review uniqueness.

ADR-005: Immutable local-day decisions and engagement ledgers

- Status: Proposed - schedule policy approval pending - Date: 2026-08-01 - Owners: Progress, streaks, rewards and refreshes

Context

Legacy state stores mutable streak counters, freeze counts/date arrays and daily refresh counters, making repair and timezone behavior ambiguous.

Decision

Store event instants in UTC plus the immutable IANA timezone and local date used for each daily decision. `streak_days` records one finalized qualifying, frozen or missed closed day with evidence. `reward_ledger` stores grants, redemptions and expirations. Refresh usage is the count of accepted `plan_refreshes`.

Consequences and verification

Current/longest streak and balances are reproducible and repairable from facts. The current day may be provisional but not finalized early. Property tests cover midnight, DST gaps/folds, travel/timezone changes, delayed events, freeze races and ledger reconciliation.

ADR-006: Typed conversations with normalized messages

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: Conversations and check-ins; Symptom observations

Context

Legacy chat, care-plan, symptom and weekly systems duplicate raw message arrays, threads and summaries. LangGraph checkpoint state can be mistaken for history.

Decision

Use typed conversations, ordered replay-safe conversation_messages and versioned summaries with an explicit covered-through message ID. Weekly check-ins use versioned questions/responses and may link to a conversation. Structured symptom observations are separate domain facts. LangGraph checkpoints remain expiring vendor runtime state only.

Consequences and verification

Conversation type replaces separate table families while preserving behavior. Deletion and retention cover messages, summaries and checkpoints. Tests cover message retry/order, summary coverage/factuality, type isolation, checkpoint loss and reconstruction from canonical history.

ADR-007: PostgreSQL-authoritative jobs, outbox and idempotency

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: Media, jobs and operational infrastructure

Context

In-process background tasks can disappear during restart and at-least-once queue delivery can duplicate business effects.

Decision

Every asynchronous business operation starts with a committed `ops.generation_jobs` row. Redis transports identifiers only. Workers use leases, heartbeats, bounded attempts, retry wait and dead-letter states. Domain changes and outbox rows commit together. Every mutation uses an `actor/operation/ Idempotency-Key` record with request hash and replayable result.

Consequences and verification

Redis loss delays work without losing truth; duplicate delivery is safe. Provider calls happen outside transactions. Worker kill/restart, lease stealing, timeout, duplicate message, conflicting request hash and dead-letter tests are mandatory.

ADR-008: Versioned AI gateway, evidence and fail-closed safety

- Status: Proposed - provider/model and clinical approval pending - Date: 2026-08-01 - Owners: Evidence, AI safety and evaluation

Context

Provider calls and prompts are scattered, model fallbacks vary, and health-facing output requires traceable evidence and safety review.

Decision

Provider SDKs exist only behind gateway adapters. Prompts are source-controlled by task, version, output schema, model policy and safety policy. Generation uses structured output, authoritative evidence, citation validation and deterministic plus evaluated clinical-safety gates. Unsafe, unverifiable or structurally invalid content fails before publication. Medical-persona/diagnostic claims are prohibited. Invocation telemetry stores metadata/cost/latency/status, not long-term raw health prompts.

Consequences and verification

Primary/fallback selection is replaceable and benchmarked in staging after key rotation/privacy review. Golden, adversarial, red-flag, provider-timeout and cost tests gate every prompt/model version.

ADR-009: Immutable permanent media before plan publication

- Status: Proposed - provider selection pending - Date: 2026-08-01 - Owners: Media, jobs and operational infrastructure

Context

Temporary/broken URLs and usage arrays cannot support the four-action/sixteen- image product contract or reliable deletion/inventory.

Decision

media_assets represents immutable object versions identified by content hash, storage key, media type, dimensions, safety/approval state and lifecycle. Items and variants reference assets by foreign key. Provider output is copied to permanent storage and verified before publication. If both providers fail, use an approved permanent category fallback or fail the job.

Consequences and verification

Usage is derived from foreign keys; duplicate/orphan/missing objects can be reconciled. Tests cover hash deduplication, reachability, provider outage, fallback category, object deletion and the sixteen-image invariant.

ADR-010: Versioned metric catalog over canonical facts

- Status: Proposed - formula owner approval pending - Date: 2026-08-01 - Owners: Domain metric owners; Data/observability steward

Context

Duplicate counters, arrays and scattered functions allow multiple values to claim the same product or reliability concept.

Decision

Every metric record includes formula, grain, timezone, exclusions, canonical source fields, owner, freshness and consumers. Current-day daily metrics are provisional until local day close. Caches/projections require lineage, refresh policy and reconciliation tests. SLOs reference catalog IDs and telemetry.

Consequences and verification

Metric changes become reviewed architecture changes rather than hidden code edits. Formula property tests, canonical-vs-projection reconciliation and catalog consumer checks gate release. Initial thresholds remain targets until dated production-readiness evidence exists.

ADR-011: OpenAPI v2, token-derived ownership and RFC 9457 errors

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: API architecture; Domain owners

Context

The legacy client duplicates URL/token/DTO logic and v1 routes include user IDs, inconsistent errors and mutation semantics.

Decision

Check in OpenAPI 3.1.1 under /api/v2 with stable operation IDs. Private identity comes only from verified tokens. Every mutation requires Idempotency-Key; plan and other versioned mutations use If-Match. Errors use RFC 9457 plus stable code and correlation ID. v1 is absent from final production.

Consequences and verification

The generated TypeScript client becomes the mobile network contract. Runtime, OpenAPI, examples and generated-client hashes must match in CI. Authorization, problem-details, idempotency and stale-revision contract tests cover every operation.

ADR-012: Clean PostgreSQL 17 root and shadow-database migration

- Status: Proposed - migration owner approval pending - Date: 2026-08-01 - Owners: Platform engineering; Data migration owner

Context

Historical migration branches are not a safe v2 foundation and the only legacy database must not be restructured in place.

Decision

Create one clean v2 Alembic root for private app/ops schemas on PostgreSQL 17; keep vendor checkpoint migrations isolated. Restore legacy backup only into an isolated source. Versioned dry-run/apply migration writes a fresh target using explicit legacy-ID-to-UUID mappings and per-row disposition. Rehearse twice.

Consequences and verification

Cutover avoids dual-write/v1 compatibility. Rollback after v2 writes is only to compatible v2 or restore. Blank bootstrap, upgrade/rollback, drift, 1,022-row reconciliation, object hash classification and identical-rehearsal evidence gate release.

ADR-013: Purpose-limited health-data lifecycle and complete erasure

- Status: Proposed - legal/clinical/owner decisions pending - Date: 2026-08-01 - Owners: Privacy owner; Onboarding, consent and profiles

Context

Assessments, symptoms, conversations, plans and summaries are health-related and span identity, DB, objects, runtime/cache and providers. Exact jurisdiction, retention and product classification are unresolved.

Decision

Collect only purpose/consent-versioned data, encrypt and owner-bind it, omit it from production telemetry and include it in export/erasure. Deletion is a durable, restartable job covering Firebase, PostgreSQL, storage, conversations, summaries, checkpoints and caches, followed by a verification enumerator. No compliance or diagnostic claim is made without approved evidence.

Consequences and verification

Public release blocks on recorded legal/clinical decisions. Tests prove consent versioning, access, export, partial-failure recovery and zero disallowed remnants. Any legal retention exception must be minimal and explicitly approved.

ADR-014: Redacted OpenTelemetry and evidence-linked operations

- Status: Proposed - telemetry platform selection pending - Date: 2026-08-01 - Owners: Platform engineering; Security/privacy owner

Context

Production decisions require user-relevant SLIs without leaking health answers, prompts, emails, tokens or conversation content.

Decision

Emit vendor-neutral OpenTelemetry traces, metrics and structured logs through an allowlisted attribute schema. Correlation IDs and stable error/job/provider codes link signals. Each SLO maps to a metric catalog record; every alert maps to a runbook. Audit events are content-free security/account facts. Release claims require dated evidence.

Consequences and verification

Debugging relies on state/codes rather than sensitive payloads. Automated canary scans inject recognizable secret/health strings and fail if any appear in logs, traces or errors. Staging exercises alerts and runbooks with an operator other than the author.

ADR-015: Generated-client, feature-aligned iOS/Android mobile architecture

- Status: Proposed - written owner approval pending - Date: 2026-08-01 - Owners: Mobile engineering

Context

The legacy app has large screens, raw fetch/token/URL duplication, AsyncStorage workflow and sensitive data, duplicate DTOs, unused Expo Router and web scope.

Decision

Support iOS/Android with React Navigation. Organize core plus backend-aligned features. One OpenAPI-generated TypeScript client owns URL, token and transport. TanStack Query owns server state; feature reducers/state machines own ephemeral workflows. Firebase owns session truth. SecureStore holds necessary credentials, never passwords/health drafts. One PlanImage uses expo-image and prefetched hero images.

Consequences and verification

Logout/account switch must purge query and UID-scoped local state. Raw fetch, direct AsyncStorage health data, duplicate DTOs and web/router dependencies fail CI. Component and Maestro tests cover both platforms, offline/process death, account switching, image readiness and accessibility.

API and database incident runbook

Status: TARGET PLANNED. Execute only in an approved environment with an identified incident owner.

1. Record deploy image digest, UTC time, request IDs and affected endpoint class. Redact health content, authorization headers and tokens. 2. Read readiness and dependency health; compare error/latency to the approved baseline. Verify database connection saturation, migration revision and Redis availability without exposing data. 3. If a migration/deploy correlates with the incident, stop promotion and use the tested compatible v2 rollback/restore path. Legacy is never a post-write rollback target. 4. For database pressure, capture read-only lock/connection evidence. Do not kill sessions or alter schema without DBA/owner approval. 5. Verify recovery with an authenticated, non-sensitive smoke test and record the exact evidence. Create follow-up for missing alert/runbook/test coverage.

Jobs, providers and media runbook

Status: TARGET PLANNED. PostgreSQL job state is authoritative; Redis only transports identifiers.

1. Inspect job ID, idempotency key, state, attempts, lease owner/expiry, heartbeat and redacted provider error classification.
2. Check provider health and credential rotation state. Do not paste prompts, user content, secrets or generated health advice into operational tooling.
3. A worker may retry only according to the recorded bounded policy. Expired leases may be reclaimed through the tested lease mechanism; never invoke a provider while a database transaction is open.
4. Before publication, prove the plan has exactly four valid actions and sixteen reachable, permanent, approved assets. Missing/temporary/blank media leaves the plan unpublished.
5. Escalate exhausted, unsafe or unverifiable output to dead letter and the relevant product/safety owner. Record a replay decision and evidence.

Restore and migration runbook

Status: blocked until an isolated PostgreSQL 17 restore is evidenced.

1. Use an isolated PostgreSQL 17 source and fresh v2 target. Never restore a dump into a serving cluster. 2. Preserve source hashes and run the documented restore procedure. Record DDL and COPY success plus read-only counts. 3. Reconcile exactly 1,022 application COPY rows; record the separate one-row alembic_version entry. This inventory does not itself prove restoration. 4. Run two versioned dry-run/apply rehearsals into fresh targets. Compare row mapping, quarantine, foreign-key/key and target reconciliation outputs. 5. The supplied storage ZIP is a valid empty 22-byte archive: classify every expected object as missing. Regenerate retained assets, permanently store and hash them, then prove reachability before READY publication. 6. Stop on any mismatch. PG17 restore remains unverified in this package because local clients are PG14 and Docker was unavailable when evidence was recorded.

Security, privacy, export and erasure runbook

Status: TARGET PLANNED. Legal jurisdiction, retention and clinical escalation wording remain owner decisions.

1. Treat a suspected exposure as an incident: limit access, preserve redacted evidence, identify the environment and escalate to the security/privacy owner. Do not expand access to investigate. 2. Verify identity from a token-derived internal user and repository owner predicate. A body user_id or client state does not authorize access. 3. For export/erasure, record request identity, legal hold decision, approved scope and idempotency key. The target scope includes Firebase, PostgreSQL, object storage, conversations, summaries, checkpoints, caches and derived artifacts. 4. Confirm each scope with durable completion evidence. Failed deletion is not silently retried beyond the recorded job policy; escalate with the failed scope and redacted reason. 5. Close only after owner review and dated evidence; never infer compliance from code or a dashboard.

Under-pressure index

Scope: TARGET PLANNED operating model. It is not evidence that the target worker, alerts, dashboards or controls are deployed. Do not improvise writes in legacy production.

First five minutes

1. Declare an incident owner and record UTC start time, user impact and known boundaries. Do not put health content, tokens or prompt text in the ticket. 2. Read the relevant runbook below. Start with read-only checks and preserve logs/evidence before retrying or restarting work. 3. Confirm environment, deployed immutable image, migration revision, provider status and whether the affected data is legacy or v2. 4. If cross-user exposure, suspected credential exposure, deletion failure or unsafe health content is possible: restrict access, preserve evidence and escalate to security/privacy/clinical owner. Do not make a compliance claim.

Situation index

Signal	First check	Runbook	Unsafe shortcut to avoid
API unavailable/error surge	Readiness, DB/Redis connectivity, deploy change	API_AND_DATABASE.md	Restart loops or disabling auth
Job backlog/retries	Job state, lease age, provider error class	JOBS_AND_PROVIDERS.md	Requeueing all work blindly
Plan missing/broken image	Plan status and asset reachability	JOBS_AND_PROVIDERS.md	Marking READY without 16 assets
Wrong account/data access	Request/audit scope and token subject	SECURITY_PRIVACY.md	Querying another user's records
Deletion/export request stuck	Erasure job evidence by scope	SECURITY_PRIVACY.md	Calling providers without case record
Restore/migration issue	PG17 environment and count reconciliation	RESTORE_AND_MIGRATION.md	Restoring into serving database

Exit discipline

Document timestamps, commands/queries, immutable artifact identifiers, impact, decision owner and follow-up. An alert is closed only after user impact is understood and the linked acceptance evidence is attached.

Legacy migration evidence register

Evidence state: Partial; restore rehearsal not verified Release effect: Blocking for Phase 0 restore gate and Phase 5 migration gate

Database backup counts

The supplied SQL backup's COPY sections contain:

Table	COPY rows
daily_assignments	145
question_sessions	4
recommendation_advices	498
recommendation_completions	1
recommendation_records	169
recommendation_redistributions	0
recommendation_schedules	141
schedule_redistributions	5
session_processing_status	8
user_profiles	25
user_responses	25
user_schedules	1
Application total	1,022
alembic_version (not an application row)	1

These counts prove the COPY-section inventory, not a successful restore or v2 migration.

Storage archive

The supplied storage ZIP is a structurally valid but empty 22-byte archive; unzip -t identifies it as empty. Therefore:

- no legacy media object can be restored; - a migration report must classify expected legacy object references as missing; - retained plan imagery must be regenerated using an approved provider or permanent category fallback; and - each regenerated object must be permanently stored, hashed, safety-approved and linked before the plan can become READY.

Unverified restore blocker

The available local pg_restore and psql tools are PostgreSQL 14, not 17, and the Docker daemon is unavailable. The documented isolated PostgreSQL 17 restore has therefore not run. Required closure evidence is:

1. isolated PostgreSQL 17 source restore logs with application DDL/COPY success; 2. read-only per-table counts matching the inventory above; 3. two versioned dry-run/apply rehearsals into fresh v2 targets; 4. identical row mappings/quarantine outputs and target reconciliation; and 5. regenerated asset reachability/hash evidence.

Until all five exist, do not label backup restoration, data migration, media migration or production cutover ready.